DEEP LEARNING
UNIT – II COMPLETE ANSWERS

Part A – 2 Marks | Part B – Expanded 10-Mark Format

Unit focus	Feedforward Neural Networks, MLP, Activation & Loss Functions, Backpropagation & Regularization
Part A	10 concise, exam-ready answers
Part B	8 expanded answers with headings, equations, tables and relevant diagrams
Scope	Part A and Part B only. Part C numerical exercise is not included.

B.Tech V Semester – Artificial Intelligence & Machine Learning

UNIT – II | PART A: SHORT ANSWERS (2 MARKS EACH)

Q1. Define Multi-Layer Perceptron (MLP). How does it overcome the fundamental limitations of a single-layer perceptron?

A Multi-Layer Perceptron (MLP) is a feedforward neural network consisting of an input layer, one or more hidden layers and an output layer. Neurons in successive layers are connected by weighted links and use activation functions.

- Hidden layers with non-linear activation functions allow an MLP to learn complex, non-linearly separable patterns such as XOR.
- A single-layer perceptron can form only a linear decision boundary; an MLP can form more complex decision boundaries.

Q2. What is a Feed Forward Neural Network (FFNN)? State the direction of data flow.

A Feed Forward Neural Network (FFNN) is an artificial neural network in which information moves in one direction only, from the input layer through hidden layers to the output layer.

- There are no feedback connections or cycles.
- It computes an output using weights, biases and activation functions.

Q3. List any three activation functions used in hidden layers (Sigmoid, Tanh, ReLU) with their mathematical expressions and output ranges.

Three commonly used activation functions are:

- Sigmoid: σ(x) = 1 / (1 + e■■); range: (0, 1).
- Tanh: tanh(x) = (e■ – e■■) / (e■ + e■■); range: (–1, 1).
- ReLU: f(x) = max(0, x); range: [0, ∞).

Q4. What are the three variants of Gradient Descent (Batch GD, Stochastic GD, Mini-batch GD)? Compare their convergence speed.

Gradient Descent updates model parameters in the direction that reduces the loss function. Its three variants are:

- Batch GD: uses the entire dataset for one update; stable but slower per update.
- Stochastic GD: uses one training example per update; updates are fast but noisy.
- Mini-batch GD: uses a small group of examples; combines efficient computation with relatively stable convergence.

Q5. What are the common Loss / Cost functions used in Deep Learning (MSE, Binary Cross-Entropy, Categorical Cross-Entropy) and their applications?

A loss function measures the difference between predicted output and the actual target.

- MSE: L = (1/n)Σ(y – ■)²; commonly used for regression.
- Binary Cross-Entropy: L = –[y log(■) + (1–y)log(1–■)]; used for binary classification.
- Categorical Cross-Entropy: L = –Σ y■ log(■■); used for multi-class classification.

Q6. Define Overfitting and Underfitting in neural network training.

Overfitting occurs when a model learns training data too closely, including noise, and performs poorly on unseen data. It has low training error but high test/validation error.

- Underfitting occurs when a model is too simple or insufficiently trained to capture data patterns.
- It generally has high error on both training and unseen data.

Q7. Define Regularization and list different types of regularization methods used in deep learning.

Regularization is a technique used to reduce overfitting and improve a model's ability to generalize to unseen data.

- Methods include L1 regularization (Lasso), L2 regularization (Ridge/weight decay), dropout, early stopping and data augmentation.
- They constrain model complexity or reduce dependence on particular training examples.

Q8. Define Empirical Risk Minimization (ERM) and state its mathematical formulation.

Empirical Risk Minimization (ERM) selects model parameters that minimize the average loss over the available training dataset.

- R■(θ) = (1/n) Σ■■■■ L(f(x■; θ), y■).
- The objective is θ* = arg minθ R■(θ).

Q9. Explain the Vanishing and Exploding Gradient Problem in deep neural networks.

During backpropagation, gradients are repeatedly multiplied across layers. If they become very small, the vanishing gradient problem occurs; if they become very large, the exploding gradient problem occurs.

- Vanishing gradients make early layers learn very slowly.
- Exploding gradients cause unstable, excessively large weight updates.

Q10. What is Greedy Layer-Wise Pre-training and why was it historically essential for deep architectures?

Greedy Layer-Wise Pre-training trains a deep network one layer at a time, usually in an unsupervised manner, before fine-tuning the complete network.

- Each layer learns a representation from the previous layer using methods such as autoencoders or RBMs.
- Historically, it helped initialize deep networks and reduce optimization difficulties when data and computing resources were limited.

UNIT – II | PART B: DETAILED 10-MARK ANSWERS

Expanded exam answers with main points, derivations, formulas and relevant labelled diagrams.

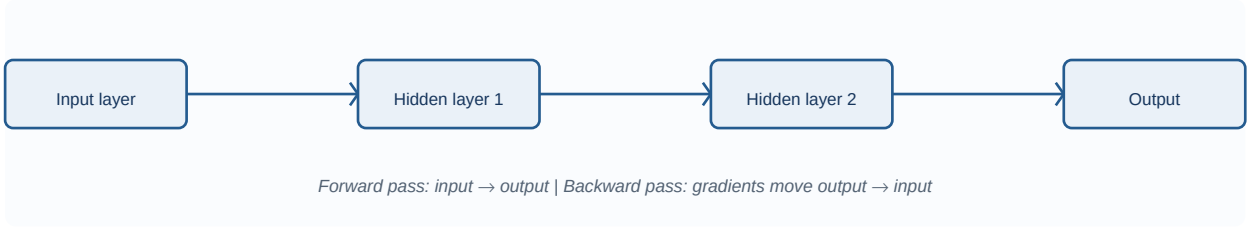
Writing tip: For a 10-mark answer, write the introduction, explain each numbered heading, include the diagram and equations asked in the question, and end with a conclusion.

Q1. Explain the Backpropagation Algorithm in detail for training a Multi-Layer Perceptron. Derive the error propagation equations using the chain rule of differentiation.

1. Introduction

Backpropagation is a supervised learning algorithm used to train a Multi-Layer Perceptron (MLP). It calculates the contribution of each weight and bias to the output error. The error is propagated from the output layer towards the input layer, and parameters are updated using Gradient Descent or another optimizer.

2. MLP architecture and principle



An MLP has an input layer, one or more hidden layers and an output layer. Forward propagation calculates the prediction. Backpropagation calculates gradients that indicate how the parameters should change to reduce the loss.

3. Steps of the algorithm

- Initialize weights and biases with suitable small values; select learning rate η.
- Forward pass: calculate weighted sums and activations layer by layer until prediction ■ is obtained.
- Loss calculation: compare prediction ■ with actual target y using a suitable loss function.
- Output error: calculate the derivative of loss with respect to the output layer.
- Backward pass: propagate error terms to hidden layers using the chain rule.
- Gradient calculation: calculate derivatives for every weight and bias.
- Parameter update: subtract learning-rate-scaled gradients from the parameters.
- Repeat for all mini-batches and epochs until loss converges or a stopping condition is reached.

4. Mathematical derivation using the chain rule

For neuron i in layer l, the weighted input and activation are:

```
z■■■■■ = Σ■ w■■■■■■ a■■■■■^(l−1)■ + b■■■■■
a■■■■■ = ϕ(z■■■■■)
```

Here, w represents weights, b is bias, ϕ is the activation function and a is the output of the previous layer.

Output-layer error term: For output neuron k, the chain rule gives:

```
δ■■■■■ = ∂L/∂z■■■■■ = (∂L/∂a■■■■■) ϕ′(z■■■■■)
```

Hidden-layer error term: The error at a hidden neuron is calculated from the weighted errors in the next layer:

```
δ■■■■■ = ϕ′(z■■■■■) Σ■ w■■■■■■^(l+1)■ δ■■■■■^(l+1)■
```

Gradients of weight and bias:

```
∂L/∂w■■■■■■ = δ■■■■■ a■■■■■^(l−1)■
∂L/∂b■■■■■ = δ■■■■■
```

5. Gradient Descent update rules

```
w■■(new) = w■■(old) − η (∂L/∂w■■)
b■(new) = b■(old) − η (∂L/∂b■)
```

The learning rate η controls the size of the update. A very high learning rate can make training unstable; a very low learning rate can make training slow.

6. Advantages and limitations

- Efficiently calculates gradients for networks with many layers and parameters.
- Works with differentiable activation and loss functions.
- Forms the basis of training most feedforward neural networks.
- Vanishing or exploding gradients can make training difficult; suitable activations, initialization and optimization methods are important.

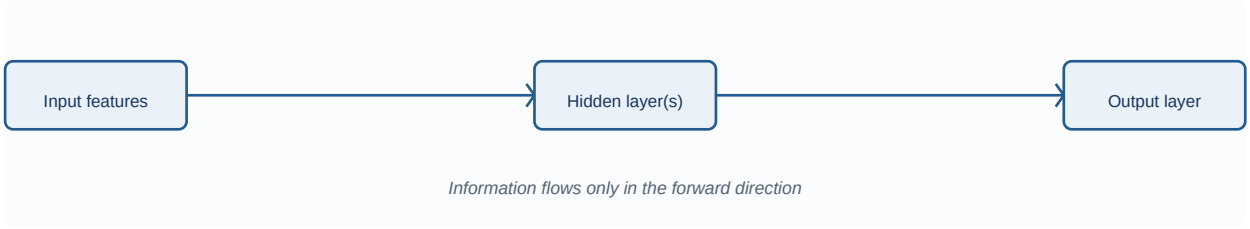
Conclusion: Backpropagation applies the chain rule to calculate gradients from output to input. Gradient-based updates reduce the loss and enable the MLP to learn from training examples.

Q2. What is a Feed Forward Neural Network? Explain gradient-based learning, loss computation, and parameter optimization in FFNNs.

1. Definition

A Feed Forward Neural Network (FFNN) is an artificial neural network in which information flows in one direction only: from the input layer, through hidden layers, to the output layer. It has no feedback connections or cycles. FFNNs are used for classification and regression.

2. Architecture



- Input layer receives the numerical features of an example.
- Hidden layers transform the input using weights, biases and activation functions.
- Output layer produces the final prediction, such as a class probability or a numerical value.

3. Forward propagation

Each layer calculates a weighted sum, adds a bias and applies an activation function:

```
z■■■ = W■■■ a■■■¹■ + b■■■
a■■■ = ϕ(z■■■)
```

The activation of one layer becomes the input to the next layer. The final activation is the predicted output ■.

4. Loss computation

A loss function measures the difference between predicted output and actual target. The loss function depends on the task:

- Mean Squared Error (MSE): commonly used for regression.
- Binary Cross-Entropy: used for binary classification.
- Categorical Cross-Entropy: used for multi-class classification.

```
MSE = (1/n) Σ■ (y■ − ■■)²
```

5. Gradient-based learning

The network learns by changing weights and biases to reduce the loss. Backpropagation calculates partial derivatives of the loss with respect to each parameter. The gradients indicate the direction and rate at which the loss changes.

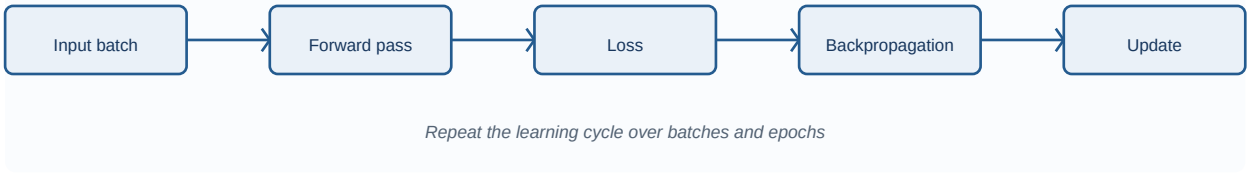
```
Weight gradient = ∂L/∂W
```

6. Parameter optimization using Gradient Descent

```
W ← W − η (∂L/∂W)
b ← b − η (∂L/∂b)
```

Here η is the learning rate. Common variants are Batch Gradient Descent, Stochastic Gradient Descent and Mini-batch Gradient Descent.

7. Complete training cycle



- Initialize weights and biases.
- Feed the training data into the network.
- Calculate predictions using forward propagation.
- Compute loss using the selected cost function.
- Apply backpropagation to calculate gradients.
- Update weights and biases and repeat the process.

Conclusion: An FFNN learns through forward propagation, loss computation, backpropagation and parameter optimization. Repeated updates gradually improve prediction accuracy.

Q3. Explain the difficulties and challenges in training Deep Neural Networks (vanishing gradients, exploding gradients, saddle points, saturation of activation functions, overfitting).

Introduction: Deep Neural Networks can learn complex patterns, but their training is affected by gradient behavior, complex loss surfaces and the risk of memorizing training data.

1. Vanishing Gradient Problem

During backpropagation, gradients are multiplied across many layers. If derivatives are repeatedly smaller than one, the gradient becomes extremely small as it moves towards earlier layers. Initial layers then receive very little learning signal and update their weights very slowly.

- Often associated with deep networks using saturating activations such as sigmoid and tanh.
- Earlier layers may learn slowly or stop learning effectively.
- Possible remedies: ReLU-family activations, suitable weight initialization and architectures that improve gradient flow.

2. Exploding Gradient Problem

In some networks, repeated multiplication makes gradients extremely large. This causes very large weight updates, unstable training and sometimes numerical overflow.

- Loss may fluctuate sharply or become NaN/infinite.
- Gradient clipping limits the magnitude of gradients.
- Careful initialization and suitable learning rates also help.

3. Saddle Points

A saddle point is a point on the loss surface where the gradient is zero or nearly zero, but it is not a local minimum. The loss increases in some directions and decreases in other directions. Optimization can slow down near such flat regions.

- Saddle points are common in high-dimensional parameter spaces.
- They can slow optimization even when a good minimum has not been reached.
- Momentum and adaptive optimizers can help move through flat regions.

4. Saturation of Activation Functions

Sigmoid and tanh become saturated for very large positive or negative inputs. In these flat regions their derivatives are close to zero. During backpropagation this weakens the gradient signal and slows learning.

```
Sigmoid: σ(x) = 1 / (1 + e■■)
Tanh: tanh(x) = (e■ – e■■) / (e■ + e■■)
```

ReLU and its variants are commonly used in hidden layers to reduce saturation-related gradient problems.

5. Overfitting

Overfitting occurs when a model learns training examples too closely, including noise and irrelevant details. It performs well on training data but poorly on unseen validation or test data.

- Causes: excessive model complexity, insufficient training data and too many training epochs.
- Remedies: dropout, L1/L2 regularization, early stopping and data augmentation.
- Use separate validation and test datasets to check generalization.

6. Comparison of challenges

Challenge	Main effect	Possible remedy
Vanishing gradient	Very small gradients; early layers learn slowly	ReLU-family activations; suitable initialization
Exploding gradient	Very large updates; unstable training	Gradient clipping; careful initialization
Saddle point	Slow progress in flat regions	Momentum; adaptive optimizers
Saturation	Derivatives close to zero	Non-saturating hidden activations
Overfitting	Poor performance on unseen data	Regularization; dropout; early stopping

Conclusion: Stable deep-network training requires suitable activations, initialization and optimizers, gradient control and regularization to improve generalization.

Q4. Explain various Regularization techniques in Deep Learning (L1 Lasso, L2 Ridge / Weight Decay, Early Stopping, Data Augmentation) and explain how they reduce overfitting.

1. Introduction and need

Regularization refers to techniques that reduce overfitting and improve performance on unseen data. A model with too many effective parameters may memorize training examples. Regularization discourages unnecessary complexity or exposes the model to more varied examples.

2. L1 Regularization (Lasso)

L1 adds the sum of absolute weight values to the original data loss. It encourages some weights to become zero or close to zero, producing a sparse model.

```
L_total = L_data + λ Σ■ |w■|
```

- λ is the regularization strength.
- It can reduce the number of effective features or connections.
- If λ is too large, the model may become too simple and underfit.

3. L2 Regularization (Ridge / Weight Decay)

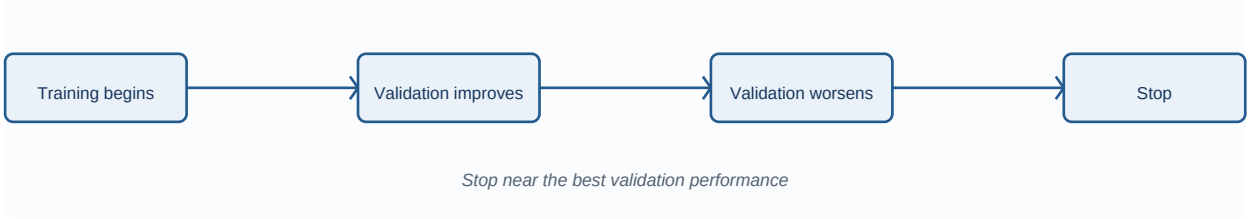
L2 adds the sum of squared weights to the loss. It penalizes large weights and encourages smoother parameter values.

```
L_total = L_data + λ Σ■ w■²
```

- Discourages excessively large parameter values.
- Usually shrinks weights rather than forcing many weights to exactly zero.
- Weight decay is a commonly used implementation of this idea.

4. Early Stopping

Training loss may continue to decrease while validation loss begins to increase. This indicates that the model may be overfitting. Early stopping monitors validation performance and stops training when it no longer improves for a selected patience period.



5. Data Augmentation

Data augmentation increases training-data diversity by applying realistic transformations to existing examples. It helps the model learn general patterns instead of memorizing exact training samples.

- Image examples: horizontal flipping, cropping, rotation, scaling and brightness changes.

- Text examples: carefully selected synonym replacement or paraphrasing, provided the label meaning is preserved.
- Augmentations must be appropriate for the task and must not change the correct label.

6. Comparison of regularization methods

Method	Working principle	How it reduces overfitting
L1	Adds λΣ|w| to loss	Encourages sparse weights
L2	Adds λΣw² to loss	Penalizes large weights
Early stopping	Monitors validation loss	Stops before excessive fitting
Data augmentation	Creates varied training examples	Improves data diversity

7. Benefits and care

- Improves generalization to unseen examples.
- Can make training more robust and reduce sensitivity to noise.
- Regularization strength must be selected carefully; too much regularization can cause underfitting.

Conclusion: L1 and L2 control parameter complexity, early stopping controls training duration, and data augmentation increases data variation. Together they help reduce overfitting.

Q5. What is Empirical Risk Minimization (ERM)? Explain the trade-off between Empirical Risk and Generalization Error with an illustrative diagram.

1. Definition of ERM

Empirical Risk Minimization (ERM) is a learning principle in which model parameters are selected by minimizing the average loss over the available training dataset. It finds a model that makes the smallest average prediction error on the observed examples.

```
R■(θ) = (1/n) Σ■■■■ L(f(x■; θ), y■)
```

Here n is the number of training examples, x■ is an input, y■ is its target, f is the model, θ represents model parameters and L is the loss function.

2. ERM optimization objective

```
θ* = arg minθ R■(θ)
```

The learning algorithm searches for parameter values θ* that minimize empirical risk. Gradient-based methods can be used to perform this optimization.

3. Generalization error

Generalization is the ability of a trained model to make accurate predictions on new examples not used during training. Generalization error measures the prediction error on unseen data. Validation or test data is used to estimate this performance.

4. Trade-off between empirical risk and generalization error

- Low model complexity: the model may be too simple to capture the underlying pattern. Training error and unseen-data error can both be high; this is underfitting.
- Moderate model complexity: the model captures important patterns and provides a useful balance between training fit and generalization.
- Very high model complexity: training error may become very low, but the model can memorize noise. Generalization error may increase; this is overfitting.

5. Illustrative diagram

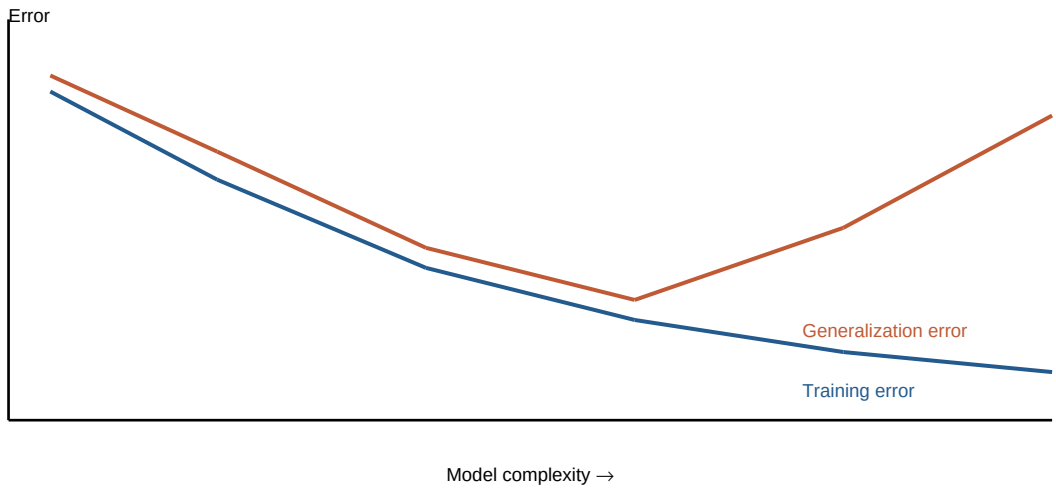


Diagram explanation: Training error generally decreases as model complexity increases. Generalization error often decreases initially, reaches a low region and then rises when the model begins to overfit.

6. Role of regularization and validation

- Regularization discourages overly complex models and helps control overfitting.
- Validation data helps compare model settings and select suitable complexity.
- The final test set should be used to evaluate performance after model selection.

Conclusion: ERM minimizes average training loss, but minimizing training error alone does not guarantee good performance on unseen data. A successful model balances empirical risk with generalization.

Q6. Derive the weight update rule for a single neuron using Gradient Descent with Mean Squared Error (MSE) loss function.

1. Single-neuron model

Consider input vector x = [x■, x■, …, x■], weights w = [w■, w■, …, w■], bias b and target y. The neuron calculates a weighted sum and applies an activation function.

```
z = w■x + b
■ = ϕ(z)
```

Here ■ is the predicted output and ϕ is the activation function. For a linear neuron, ϕ(z)=z.

2. Mean Squared Error loss

For one training example, the squared-error loss is:

```
L = ½ (y − ■)²
```

The factor ½ is included to simplify differentiation. For multiple examples, MSE is the average of squared differences.

3. Derivation of weight gradient using chain rule

For weight w■, apply the chain rule through the loss, prediction and weighted input:

```
∂L/∂w■ = (∂L/∂■)(∂■/∂z)(∂z/∂w■)
```

The individual derivatives are:

```
∂L/∂■ = ■ − y
∂■/∂z = ϕ′(z)
∂z/∂w■ = x■
```

Substituting these terms gives:

```
∂L/∂w■ = (■ − y) ϕ′(z) x■
```

4. Derivation of bias gradient

Since z = Σ■ w■x■ + b, the derivative of z with respect to b is 1.

```
∂L/∂b = (∂L/∂■)(∂■/∂z)(∂z/∂b)
∂L/∂b = (■ − y) ϕ′(z)
```

5. Gradient Descent update rules

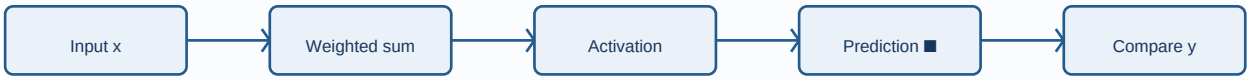
Gradient Descent updates a parameter by subtracting the learning rate multiplied by its gradient:

```
w■(new) = w■(old) − η (■ − y) ϕ′(z) x■
b(new) = b(old) − η (■ − y) ϕ′(z)
```

For a linear neuron, $\phi'(z)=1$. Therefore:

```
w■(new) = w■(old) − η (■ − y)x■
b(new) = b(old) − η (■ − y)
```

6. Interpretation and training flow



Gradients are calculated from the error and used to update weights and bias

- The prediction error determines the gradient direction.
- The learning rate η controls the size of each update.
- Repeated updates over training examples gradually reduce MSE when the learning setup is suitable.

Conclusion: The chain rule applied to MSE gives gradients for weights and bias. Gradient Descent uses these gradients to update the neuron parameters and reduce prediction error.

Q7. Explain the steps involved in Unsupervised Greedy Layer-Wise Training of Deep Neural Networks (Autoencoder / RBM stacking) and its fine-tuning phase.

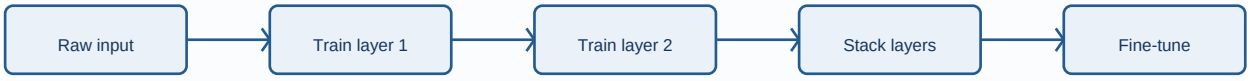
1. Introduction

Greedy Layer-Wise Training builds a deep neural network one layer at a time. Each layer is first trained to learn a useful representation of its input, usually without class labels. After pre-training, the layers are stacked and the complete network is fine-tuned for the final task.

2. Meaning of greedy and layer-wise

- Layer-wise means that one layer is trained at a time instead of training the entire deep network from the beginning.
- Greedy means each layer is trained to improve its own representation using the output of the previously trained layer.
- Unsupervised pre-training means the model learns structure from input data without requiring target labels.

3. Step-by-step training process



Unsupervised layer-wise pre-training followed by supervised fine-tuning

- Train the first autoencoder or RBM using raw input data.
- Use the hidden representation learned by the first layer as input to train the second layer.
- Repeat the process for additional layers, training one layer at a time.
- Stack the trained encoder or hidden layers to form a deep architecture.
- Add an output layer for the target task, such as classification, if labelled data is available.
- Fine-tune the complete network using supervised backpropagation so all layers adapt jointly to the final objective.

4. Autoencoder-based pre-training

An autoencoder is a neural network trained to reconstruct its input. It contains an encoder and a decoder. The encoder converts input x into hidden representation h; the decoder reconstructs the input as x■.

```
h = f(W■x + b■)
x■ = g(W■h + b■)
```

The model minimizes reconstruction loss, which measures the difference between x and x■. After training, the encoder can be retained as a feature-learning layer.



An autoencoder learns a representation and reconstructs its input

5. RBM-based pre-training

A Restricted Boltzmann Machine (RBM) is an energy-based generative model with visible and hidden units. Connections exist between visible and hidden layers, but not between units within the same layer. It learns a representation of input data by modeling its probability distribution. Trained RBM layers can be stacked.

6. Fine-tuning phase

After layer-wise pre-training, the learned layers are stacked and an output layer is added. The complete network is trained end-to-end using labelled examples. Backpropagation calculates gradients across all layers, and an optimizer updates the weights to minimize task-specific loss.

7. Advantages and limitations

- Provides useful initial weights and representations for deep networks.
- Learns progressively from lower-level to higher-level features.
- Historically helped train deep networks when optimization and computing resources were more limited.
- Requires additional pre-training stages and may not be necessary when modern initialization, activation and optimization techniques are effective.

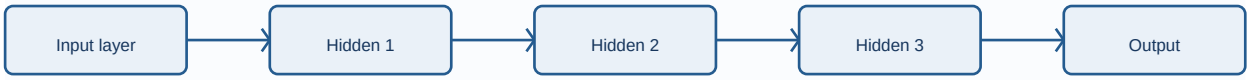
Conclusion: Greedy layer-wise training learns representations one layer at a time using autoencoders or RBMs. The stacked network is then fine-tuned end-to-end for the final task.

Q8. With a neat sketch, explain the complete architecture of a Deep Neural Network (DNN) and its forward pass, loss computation, and backpropagation mechanisms.

1. Introduction

A Deep Neural Network (DNN) is an artificial neural network containing an input layer, multiple hidden layers and an output layer. It learns complex patterns by transforming input data through several layers of weighted computations and non-linear activation functions.

2. Complete DNN architecture



A DNN contains multiple hidden layers between input and output

- Input layer receives the numerical feature vector.
- Hidden layers learn and transform intermediate representations.
- Output layer produces the final prediction, such as a class score, probability or regression value.

3. Forward pass

In forward propagation, data moves from the input layer towards the output layer. Each layer computes a weighted sum, adds bias and applies an activation function:

```
z■■■ = W■■■a■■■[1]■ + b■■■
a■■■ = ϕ(z■■■)
```

W is the weight matrix, b is the bias vector, ϕ is the activation function and a is the layer activation. The final layer produces prediction ■.

4. Output layer and activation

- For binary classification, sigmoid can represent the probability of the positive class.
- For multi-class classification, softmax converts output scores into class probabilities.
- For regression, a linear output is commonly used.

5. Loss computation

The loss function measures the difference between the prediction and the actual target. MSE is commonly used for regression, while cross-entropy is commonly used for classification.

```
L = L(y, ■)
```

The objective of training is to minimize this loss over training examples.

6. Backpropagation mechanism

Backpropagation sends error information from the output layer towards earlier hidden layers. It uses the chain rule to calculate the gradient of loss with respect to every weight and bias.

```
δ■■■ = (∂L/∂a■■■) ■ ϕ′(z■■■)
δ■■■ = ((W■■■¹■)■ δ■■■¹■) ■ ϕ′(z■■■)
∂L/∂W■■■ = δ■■■ (a■■■¹■)■
```

The symbol ■ represents element-wise multiplication. These equations calculate the error term and weight gradient for each layer.

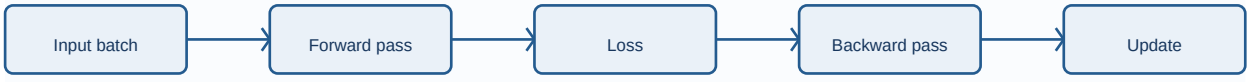
7. Parameter optimization

After gradients are calculated, the optimizer updates weights and biases. In basic Gradient Descent:

```
W■■■ ← W■■■ – η (∂L/∂W■■■)
b■■■ ← b■■■ – η (∂L/∂b■■■)
```

The process is repeated for multiple batches and epochs until the network reaches satisfactory performance or a stopping condition is met.

8. Complete training cycle



The complete DNN learning cycle repeats across training batches

- Provide input features and perform forward propagation.
- Calculate the predicted output and compare it with the true target.
- Compute loss using an appropriate cost function.
- Apply backpropagation to calculate gradients for all trainable parameters.
- Update weights and biases using an optimizer.
- Repeat training and monitor validation performance to check generalization.

Conclusion: A DNN learns through repeated forward propagation, loss computation, backpropagation and parameter optimization. Multiple hidden layers allow it to learn complex representations from data.

FINAL EXAM PRESENTATION CHECKLIST

- Draw diagrams neatly and label every layer, arrow and component.
- Underline key terms such as forward propagation, chain rule, gradient, regularization and generalization.
- Write each important equation separately and define symbols at least once.
- Use comparison tables where appropriate.
- For a 10-mark answer, explain each heading in sentences; do not write only formula names or bullet titles.